

OpenFOAM Extensions

Walter Dal'Maz Silva

Contents

1	OpenFOAM extensions	5
1.1	Compilation methods	5
1.2	Running tests	5
2	Lagrangian	7
2.1	Parcels	7
2.1.1	Extended Thermo Parcel	7
3	Thermophysical Models	9
3.1	Thermophysical Properties	9
3.1.1	Polynomial Solid Properties	9
3.1.2	Tabulated Solid Properties	10
4	Contributing	13
4.1	Directives	13
4.2	Wishlist	13
4.3	Building the docs	13
5	Changelog	15
5.1	2026-09-16	15

1. OpenFOAM extensions

Run `Allwmake` from this directory to create the full library. For creating parts of it only, navigate to the required directory and run `Allmake` if present. Otherwise, if a `Make` directory is available, run `wmake libso`. These are the three possible library levels to be built.

1.1 Compilation methods

It is possible to build the whole extensions library or isolated features, depending on your needs. The following logic is applied throughout this sources directory:

- If a directory provides an `Allwmake` file, it allows to compile the whole tree below it; this is the case for the root directory, for instance.
- If a directory provides a subdirectory called `Make/`, then it supports the individual library build which is done by running `wmake libso` from that directory.

All builds are written to `$FOAM_USER_LIBBIN`.

1.2 Running tests

Some libraries may provide test programs under a `test/` sub-directory, and all of them are structured in the same way. The following instructions indicate how to build, run, and clean those directories.

To build and run the test suite:

```
cd test
./Allrun
```

To clean test build artifacts:

```
cd test
./Allclean
```


2. Lagrangian

2.1 Parcels

2.1.1 Extended Thermo Parcel

Provides a variant of `thermoCloud` with varying specific heat.

Important: this extension currently support a single solid phase which needs to be a `tabulatedSolid` as provided in the thermophysical properties below. This will need to be generalized at some point.

Extending a parcel requires several templates to be adapted. The following building blocks are present in the implementation of `extendedThermoCloud`:

- Parcel template class is found in [ExtendedThermoParcel](#)
- The derived parcel instantiation [extendedThermoParcel](#)
- The derived cloud instantiation [extendedThermoCloud](#)

Or in a tree view:

```
parcel
├── clouds
│   └── derived
│       └── extendedThermoCloud
│           ├── extendedThermoCloud.C
│           └── extendedThermoCloud.H
└── parcels
    ├── Templates
    │   ├── ExtendedThermoParcel
    │   │   ├── ExtendedThermoParcel.C
    │   │   ├── ExtendedThermoParcel.H
    │   │   ├── ExtendedThermoParcelI.H
    │   │   ├── ExtendedThermoParcelIO.C
    │   │   ├── ExtendedThermoParcelName.C
    │   └── ExtendedThermoParcelTrackingDataI.H
    └── derived
        └── extendedThermoParcel
            ├── extendedThermoParcel.H
            └── makeExtendedThermoParcelSubmodels.C
```

For using it, append the following to `controlDict` (it depends on extended thermophysical properties so that non-constant properties are supported):

```
libs
(
    "libextendedThermophysicalProperties.so"
    "libextendedLagrangianParcel.so"
);
```


3. Thermophysical Models

3.1 Thermophysical Properties

3.1.1 Polynomial Solid Properties

Library: thermophysicalProperties/solidProperties/polynomialSolidProperties

Extends OpenFOAM's `solidProperties` base class using `Polynomial<8>` to support temperature-dependent 8th-order polynomials density ($\rho(T)$), specific heat capacity ($C_p(T)$), and thermal conductivity (κ).

It is structured as follows:

```
├─ Make
│   └─ files
│       └─ options
├─ polynomialSolid.C
├─ polynomialSolid.H
├─ polynomialSolidI.H
└─ test
    ├── Allclean
    ├── Allrun
    ├── Make
    │   ├── files
    │   └─ options
    └─ testPolynomialSolid.C
```

In `system/controlDict` one must load the compiled library at startup:

```
libs
(
    "libpolynomialSolidProperties.so"
);
```

In `constant/physicalProperties` you can now specify `polynomialSolid` under `solids {}`:

```
solids
{
    polynomialSolid
    {
        molWeight    60.084;
        Hf            0.0;
        emissivity    0.9;

        // Polynomial coefficients for rho(T) = r0 + r1*T + r2*T^2 + ...
        rhoCoeffs     (2500 -0.1 0 0 0 0 0 0);

        // Polynomial coefficients for Cp(T) = c0 + c1*T + c2*T^2 + ...
        CpCoeffs       (800 0.5 -1e-4 0 0 0 0 0);

        // Polynomial coefficients for kappa(T) = c0 + c1*T + c2*T^2 + ...
        kappaCoeffs    (1.5 0 0 0 0 0 0 0);
    }
}
```

3.1.2 Tabulated Solid Properties

Extends OpenFOAM's `solidProperties` base class using `Function1<scalar>` to support tabulated temperature-dependent density ($\rho(T)$), specific heat capacity ($C_p(T)$), and thermal conductivity (κ).

It is structured as follows:

```

├─ Make
│   └─ files
│       └─ options
├─ tabulatedSolid.C
├─ tabulatedSolid.H
├─ tabulatedSolidI.H
└─ test
    ├── Allclean
    ├── Allrun
    ├── Make
    │   ├── files
    │   └─ options
    └─ testTabulatedSolid.C

```

In `system/controlDict` one must load the compiled library at startup:

```

libs
(
    "libtabulatedSolidProperties.so"
);

```

In `constant/physicalProperties` you can now specify `tabulatedSolid` under `solids {}`:

```

solids
{
    tabulatedSolid
    {
        molWeight    60.084;
        Hf           0.0;
        emissivity    0.9;

        // Tabulated density rho(T) [kg/m^3]
        rho
        table
        (
            (300 2500)
            (600 2450)
            (1200 2380)
        );

        // Tabulated specific heat Cp(T) [J/(kg K)]
        Cp
        table
        (
            (300 800)
            (500 950)
            (800 1100)
            (1200 1250)
        );

        // Tabulated thermal conductivity [W/(m K)]
        kappa
        table
        (
            (300 1.5)
            (500 1.5)

```

```
        (800 1.5)
        (1200 1.5)
    );
}
}
```


4. Contributing

4.1 Directives

- Comments added by this repository to explain changes performed to base libraries are tagged with `/// WWW:` so that they can be tracked in the future.
- Whenever possible/applicable, copy the original `files` list from the library you are modifying to use as a starting point. Remove or comment out files that are not implemented in this repository. Modify the base path of `LIB` to use `LIB = $(FOAM_USER_LIBBIN)/<your-library-name>` so that the path is writable.
- The same apply to `options`, which requires a few extra guidelines concerning its declared variables:
 - `EXE_INC`: keep the original list untouched, add the local include directories in the end. Sometimes the library will fail to compile because the original list was lacking something (in a library as OpenFOAM it probably worked because the missing files were sourced elsewhere during compilation and the authors never had to add them), append these in the end so that we can distinguish them from the ones already in the list.
 - `LIB_LIBS`: if the project compiles but on run-time there is a missing linking library, but proceed as for `EXE_INC` by listing the library in the end. It might be tricky to identify where the functionality comes from, maybe start by `ls $FOAM_LIBBIN` and look for possibly related files.

4.2 Wishlist

- ☐ Unification/generalization of sub-classes of `solidProperties` as a new `extendedSolidProperties`; this can then be used as a replacement to `solidProperties` in the `extendedThermoParcel` framework (possible need to generalize the cloud definition).

4.3 Building the docs

A `Makefile` is provided in the root directory: running `make` will generate a local documentation, `make publish` can be used by admins to publish to GitHub Pages, and `make clean` strips the generated outputs and intermediate files from the folder.

5. Changelog

5.1 2026-09-16

- Adapted `extendedThermoCloud` to upcast the solid object to a `tabulatedSolid` as a prototype for temperature dependent properties. Currently this supports a single species and need to be generalized in the future.
- Unified the build of all libraries under a single `Allwmake` at the sources root, following the full library standard. Added compilation instructions at multiple levels to `README.md`
- Added `extendedThermoCloud` support by extending `thermoCloud`; this extension if fact only needed working with `thermoParcel` as the physics was found to be at the parcel level. Added related library sources under `lagrangian/parcel/{cloud,parcels}`.